

Class Inheritance

extends

Ek class doosri class par bana sakte ho.

javascript

```
class Animal {
  constructor(name) { this.speed = 0; this.name = name; }
  run(speed) { this.speed = speed; console.log(`${this.name} runs at ${speed}`) }
  stop() { this.speed = 0; console.log(`${this.name} stands still`); }
}

class Rabbit extends Animal {
  hide() { console.log(`${this.name} hides!`); }
}

let rabbit = new Rabbit("White Rabbit");
rabbit.run(5);
rabbit.hide();
```

Andar se ye prototype se hi chalta hai: `Rabbit.prototype.__proto__ = Animal.prototype`.

extends ke baad koi bhi expression chal sakta hai (jaise function jo class return kare).

Method override aur **super**

Child me same naam ka method likhoge to wahi use hoga. Parent ka method bhi chalana ho to **super**:

- `super.method(...)`: parent ka method call karo.
- `super(...)`: parent ka constructor call karo (sirf constructor ke andar).

javascript

```
class Rabbit extends Animal {
  stop() {
    super.stop(); // pehle parent wala
    this.hide(); // phir apna
  }
}
```

Arrow function ka apna `super` nahi hota, wo bahar wale se leta hai. Normal function me `super` error deta hai.

Constructor override

Child class ka constructor na likho to JS khud `constructor(...args) { super(...args); }` bana deta hai.

Khud likhoge to:

`this` use karne se pehle `super(...)` call karna zaroori hai.

javascript

```
class Rabbit extends Animal {  
  constructor(name, earLength) {  
    super(name);           // pehle  
    this.earLength = earLength;  
  }  
}
```

Kyun? Child (derived) constructor khud object nahi banata, wo parent constructor se banwata hai. `super()` ke bina `this` hota hi nahi.

Advanced: overridden fields ka trick

Parent constructor me overridden **field** padhoge to parent ki hi value milegi, child ki nahi (kyunki child ke fields `super()` ke baad set hote hain). Methods me aisa nahi hota (child wala method chalta hai). Problem aaye to field ki jagah method ya getter use karo.

Advanced: `[[HomeObject]]`

`super` kaam kaise karta hai? Har method ke paas hidden `[[HomeObject]]` hota hai jo batata hai "main kis object/class me bana". `super` usi se parent dhoondhta hai. `this.__proto__` se ye nahi ho sakta (chain me infinite loop ban jaata hai).

Nateeja:

- `super` wala method ek object se doosre me copy karoge to galat parent milega.
- Object me method `method() {}` syntax se likho, `method: function() {}` se nahi, warna `super` nahi chalega.

Task se seekhi baatein

- Child constructor me `super()` bhoolna sabse common galti hai.
- `ExtendedClock` jaisi class banane ke liye `super(options)` call karke naya parameter jodo aur `start()` override karo.

Yaad rakho

1. `class Child extends Parent`.
2. Constructor me `super()` pehle, phir `this`.
3. Parent method ke liye `super.method()`.
4. Arrow function ka `this` / `super` bahar wale ka hota hai.